

BUILD-SUPABASE.md

Partie C.14 — Transactions PostgreSQL

Version : V1

Statut : Référence officielle des transactions PostgreSQL

1. Objectif

Cette section définit les règles officielles de gestion des transactions PostgreSQL dans Vitala.

Les transactions garantissent :

- la cohérence des données ;
- l'intégrité des opérations ;
- la gestion correcte des erreurs ;
- la fiabilité des traitements.

Toute opération critique impliquant plusieurs écritures doit être évaluée afin de déterminer si une transaction est nécessaire.

2. Principes directeurs

Les transactions reposent sur les principes suivants :

- Atomicité
- Cohérence
- Isolation
- Durabilité
- Simplicité
- Performance

Les propriétés ACID de PostgreSQL constituent le fondement de cette politique.

3. Atomicité

Une transaction représente une unité de travail indivisible.

Deux résultats seulement sont possibles :

- toutes les opérations réussissent ;

- aucune opération n'est conservée.

Les états intermédiaires ne doivent jamais être visibles.

4. Cas d'utilisation

Une transaction est recommandée lorsque :

- plusieurs tables sont modifiées ;
- plusieurs écritures doivent rester cohérentes ;
- une opération métier dépend de plusieurs mises à jour ;
- une erreur partielle créerait une incohérence.

Les opérations simples sur une seule table ne nécessitent pas systématiquement une transaction explicite.

5. Durée des transactions

Les transactions doivent être les plus courtes possible.

Elles ne doivent jamais :

- attendre une entrée utilisateur ;
- appeler un service externe ;
- effectuer des traitements longs.

Les traitements prolongés doivent être déplacés vers les APIs, les Edge Functions ou des tâches asynchrones.

6. Isolation

Le niveau d'isolation par défaut de PostgreSQL est utilisé sauf besoin particulier.

Toute modification du niveau d'isolation doit être :

- justifiée ;
 - documentée ;
 - validée lors de la revue d'architecture.
-

7. Gestion des erreurs

Toute erreur dans une transaction entraîne un `ROLLBACK`.

Les erreurs doivent être :

- journalisées lorsque nécessaire ;
 - retournées de manière contrôlée ;
 - traduites en messages compréhensibles au niveau des APIs.
-

8. Verrouillages

Les transactions doivent limiter les verrouillages.

Les opérations susceptibles de bloquer durablement d'autres utilisateurs doivent être évitées.

Les risques de blocage (`deadlocks`) doivent être analysés lors de la conception.

9. Performances

Les transactions doivent :

- contenir uniquement les opérations indispensables ;
- limiter les lectures et écritures inutiles ;
- éviter les traitements répétitifs.

Une transaction ne doit pas dégrader significativement les performances globales.

10. Domaines métier

Les transactions techniques assurent la cohérence des données.

Les décisions métier restent exclusivement gérées par les Domaines.

Les transactions ne doivent jamais contenir de logique fonctionnelle complexe.

11. APIs

Les APIs orchestrent les opérations nécessitant plusieurs transactions ou plusieurs domaines.

Lorsque plusieurs systèmes externes sont impliqués, des mécanismes adaptés (par exemple des workflows ou des stratégies de compensation) doivent être privilégiés plutôt qu'une transaction unique.

12. Documentation

Chaque transaction importante doit être documentée avec :

- son objectif ;
 - les tables concernées ;
 - les opérations réalisées ;
 - les risques identifiés ;
 - les stratégies de reprise en cas d'échec.
-

13. Évolution

Toute modification d'une transaction doit :

- être testée ;
 - être documentée ;
 - préserver l'intégrité des données ;
 - faire l'objet d'une revue lorsqu'elle impacte plusieurs domaines.
-

14. Tests

Les transactions critiques doivent être testées afin de vérifier :

- le succès complet ;
- le retour arrière (`ROLLBACK`) en cas d'erreur ;
- le comportement sous concurrence ;
- les performances.

Les scénarios d'échec doivent faire partie des tests automatisés.

15. Critères de conformité

Une transaction est conforme lorsque :

- elle garantit l'atomicité ;
 - elle reste courte ;
 - elle respecte les propriétés ACID ;
 - elle ne contient pas de logique métier complexe ;
 - elle est documentée ;
 - elle est couverte par des tests.
-

16. Conclusion

Les transactions PostgreSQL assurent la cohérence et la fiabilité des données de Vitala.

Leur utilisation doit rester ciblée, performante et conforme aux principes du Build Blueprint afin de garantir une plateforme robuste et évolutive.